

Workforce Analytics & IT Management — Technical Documentation

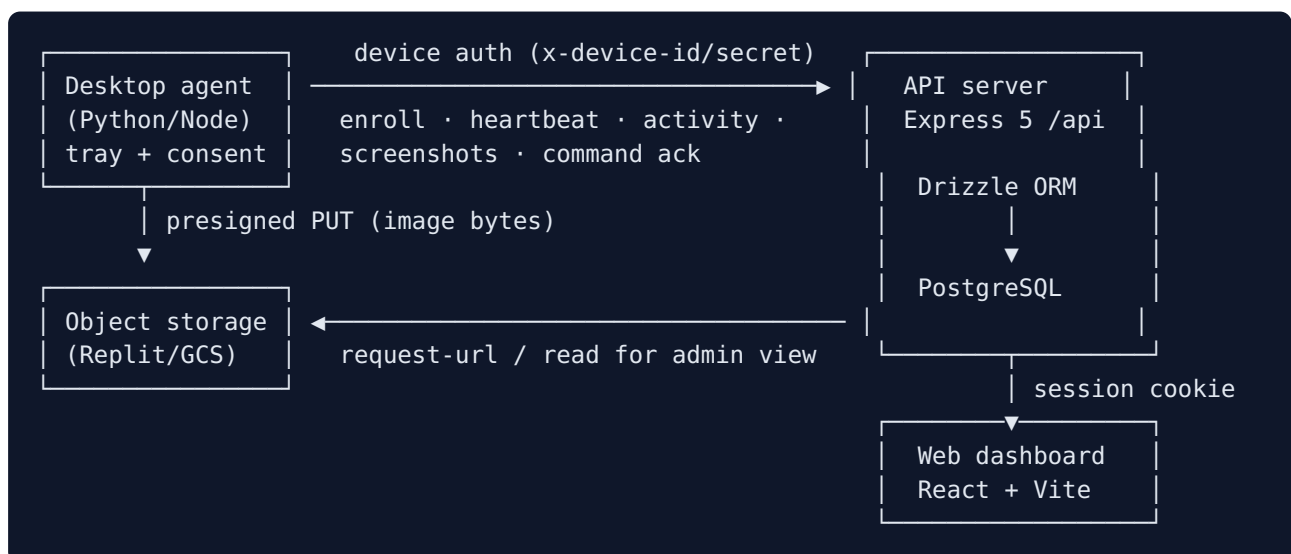
Engineering reference for the platform: architecture, stack, data model, features, build, and operations. For end-user instructions see `USER_MANUAL.md` ; for the full HTTP contract see `API.md` .

1. Overview

A **transparent, consent-based** workforce analytics and IT management platform. Enrolled devices run a visible desktop agent that — only after explicit user consent — reports foreground-app activity, idle time, and periodic screenshots, and executes authorized IT commands (lock screen, sign out). An admin-only web dashboard surfaces the data and drives configuration.

Hard product rules (do not violate): - **Transparency is mandatory** — visible tray icon, first-run consent dialog, server-enforced consent, and a notice before every screenshot. - **No fake/placeholder data** — surfaces reflect real records or an explicit empty/unavailable state. - **Permanently excluded:** keystroke/mouse capture, screen recording, video playback, live streaming.

2. Architecture



Two distinct API surfaces, two auth models:

Surface	Consumer	Auth
Sync API (/api/sync)	desktop agent	per-device headers <code>x-device-id</code> + <code>x-device-secret</code>
Admin API (everything else under /api)	web dashboard	session cookie + role <code>admin</code> / <code>super_user</code>

There is **no public/unauthenticated data endpoint**. The only open routes are `/api/healthz`, the login handshake, and the token-gated enrollment handshake.

Monorepo layout (pnpm workspaces)

```

artifacts/
  api-server/      Express API (routes, middlewares, lib)
  dashboard/       React + Vite admin UI
  mockup-sandbox/  component preview server (design only)
lib/
  db/              Drizzle schema (source of truth) + client
  api-spec/        OpenAPI contract + generated client/types (Admin API)
scripts/           create-admin, mint-enrollment-token, ...
agent/             Python desktop agent
agent-node/        Node desktop tracker client
docs/              this documentation

```

See the `pnpm-workspace` skill for workspace/TypeScript conventions.

3. Stack

- **Runtime/tooling:** Node.js 24, TypeScript 5.9, pnpm workspaces.
- **API:** Express 5.
- **DB:** PostgreSQL + Drizzle ORM; schema is the source of truth in `lib/db`.
- **Validation:** Zod (`zod/v4`), `drizzle-zod`. Sync payloads use hand-written Zod (`artifacts/api-server/src/lib/syncValidation.ts`) because the agent is an external client; the Admin API is contract-first via OpenAPI codegen.
- **Object storage:** Replit App Storage via `@google-cloud/storage`, accessed through **presigned URLs** so image bytes never transit the API.
- **Frontend:** React + Vite, wouter routing, React Query.
- **Desktop agent:** Python 3.11 (pystray, mss, Pillow, psutil, requests); a Node tracker client also exists. Both speak the same secure sync contract.

4. Authentication & authorization

User (dashboard)

- `POST /api/auth/login` validates credentials (passwords hashed with `bcrypt`), creates a session, and sets an **httpOnly** `wa_session` cookie. Login is rate-limited (`loginRateLimit` middleware).
- Sessions are persisted (`sessions` table, hashed token) and revoked on logout.
- **Roles** (`userRoleEnum`): `super_user`, `admin`, `team_member`. The **entire** admin surface is gated by `userAuth` + `requireRole("super_user","admin")` — reads included — because enrollment tokens are credentials and monitoring data is sensitive (see `routes/index.ts`).

Device (agent)

- `deviceAuth` middleware requires `x-device-id` + `x-device-secret`.
- The secret is stored **only** as a SHA-256 hash (`devices.secretHash`) and returned in plaintext **exactly once** at enrollment.
- A device with no recorded consent (`consentAcknowledgedAt` `null`) is rejected with `403`.

5. Core flows

5.1 Enrollment + consent

1. Admin mints an **enrollment token** (`enrollment_tokens`: `label`, `maxUses`, `useCount`, `expiresAt`, `revokedAt`) via the dashboard or `mint-enrollment-token`.
2. The agent collects **consent** (installer wizard on Windows; first-run dialog on macOS/Linux/manual) — full disclosure + explicit acknowledgement + the consenting person's name.
3. `POST /api/sync/enroll` with `{ token, hardwareHash, systemName, osType, consentAcknowledged: true, consentName }`. Server validates the token (not revoked/expired, `useCount < maxUses`), creates/updates the `devices` row with `consentAcknowledgedAt = now()`, and returns `{ deviceId, deviceSecret, config }`. Re-enrolling a known `hardwareHash` rotates the secret and refreshes consent.
4. Agent stores `deviceId` + `deviceSecret` locally. - **Windows install-time consent:** the Inno Setup wizard writes a one-time `enroll_seed.json`; the agent enrolls silently from it on first launch, then deletes it. This is **not** covert — installer disclosure/consent is mandatory and the agent stays visible at runtime.

5.2 Heartbeat → config + commands

`POST /api/sync/heartbeat` returns server time, the device's effective `config`, and any **pending** `device_commands`. The agent applies config and executes commands, showing an

on-screen notice first, then acknowledges via `POST /api/sync/commands/ack` (`acknowledged` → `completed / failed`).

5.3 Activity logging

The agent samples the foreground window, aggregates segments, and batch-uploads via `POST /api/sync/activity` (1-500 logs: `processName`, `windowTitle`, `startedAt`, `endedAt`, `durationSeconds`, `idleSeconds`). The server classifies each `processName`; unknown apps **auto-create an undefined `app_categories` row** for an admin to classify later (`lib/productivity.ts`).

5.4 Screenshots (presigned, 3-step)

1. `POST /api/sync/screenshots/request-url` → `{ uploadURL, storageKey }` (`/objects/uploads/<uuid>`).
2. Agent **PUTs raw bytes** directly to `uploadURL` (the URL is the short-lived credential; no device headers).
3. `POST /api/sync/screenshots` records `{ storageKey, capturedAt, fileSizeBytes }`. The server validates the `storageKey` shape it issued.

Admins view images via `GET /api/screenshots/:id/image`, an auth-gated stream; bytes never become base64 in JSON. Object ACLs live in `lib/objectAcl.ts`.

5.5 IT commands

`POST /api/devices/:id/commands` enqueues `lock_screen / logout_user` (with an optional reason). Delivered on the next heartbeat; cancellable while pending; full per-device history retained.

6. Data model (key tables)

Schema source of truth: `lib/db/src/schema/` (one file per table, barrel in `index.ts`).

Table	Purpose / key fields
<code>users</code>	dashboard accounts — <code>username</code> , <code>email</code> , <code>passwordHash</code> , <code>role</code>
<code>sessions</code>	server-side sessions — <code>userId</code> , hashed token, <code>expiresAt</code>
<code>devices</code>	enrolled machines — <code>hardwareHash</code> , <code>systemName</code> , <code>osType</code> , <code>secretHash</code> , <code>assignedUserId</code> , <code>deviceGroup</code> , <code>monitoringEnabled</code> , config fields, <code>consentName</code> , <code>consentAcknowledgedAt</code>
<code>enrollment_tokens</code>	<code>token</code> , <code>label</code> , <code>maxUses</code> , <code>useCount</code> , <code>expiresAt</code> , <code>revokedAt</code>
<code>activity_logs</code>	<code>deviceId</code> , <code>processName</code> , <code>windowTitle</code> , <code>categoryId</code> , <code>startedAt</code> , <code>endedAt</code> , <code>durationSeconds</code> , <code>idleSeconds</code>
<code>app_categories</code>	

Table	Purpose / key fields
	classification rules — pattern /app key, displayName , classification (productive/unproductive/neutral/undefined)
screenshots	deviceId , storageKey , capturedAt , fileSizeBytes , flagged
device_commands	deviceId , commandType , status (pending/acknowledged/completed/failed), reason
daily_summaries	precomputed per-day rollups
attendance_settings	global + per-device/group attendance config
shifts	working-shift definitions
timesheets	worked-time rollups (read/report layer)
projects / tasks	lightweight project + task tracking
leave_requests / leave_balances	leave workflow + per-person balances

7. Feature surface (Admin API → dashboard page)

Domain	Route prefix	Dashboard page
Auth / current user	/api/auth , /api/users	Login
Devices + config + commands + groups	/api/devices	Devices, Device Detail
Activity feed / timeline / range	/api/activity	Activity Logs, Overview
Reports (summary, leaderboard, group compare)	/api/reports	Overview
Screenshots (list, flag, image stream)	/api/screenshots	Screenshots
App classification	/api/categories	App Categories
Attendance (day/range/settings/overrides)	/api/attendance	Attendance
Timesheets	/api/timesheets	Timesheets
Projects + nested tasks	/api/projects , /api/tasks	Projects & Tasks
Shifts	/api/shifts	Shifts
Leave requests + balances	/api/leave-requests , /api/leave-balances	Leave
Enrollment tokens	/api/tokens	Enrollment Tokens

Domain	Route prefix	Dashboard page
Agent config	/api/devices/config	Agent Settings
Installer downloads	/api/downloads	Download Agent

Full request/response details for each are in `API.md`.

Agent installer downloads

GET `/api/downloads` reads the latest GitHub Releases and reports per-platform availability; GET `/api/downloads/:platform` streams the installer bytes. Asset resolution is **per-platform predicates** (not a single extension): Windows `.exe`, macOS `.dmg`, Linux = a Linux package extension (`.tar.gz` / `.tgz` / `.AppImage` / `.deb` / `.rpm`) **or an extensionless binary**, excluding the other platforms' installers and non-installer sidecars (checksums/signatures/docs). The endpoint degrades to `available:false` (never `500`) when no release or GitHub connection is present.

8. Build, run & operate

Dev commands

- `pnpm --filter @workspace/api-server run dev` — run the API (binds `PORT`, proxied at `/api`). **No watch** — restart this workflow after server changes.
- `pnpm run typecheck` — full typecheck across packages (`typecheck:libs` first if you changed `lib/*`).
- `pnpm run build` — typecheck + build all packages.
- `pnpm run test` — vitest suite (API integration tests hit the real dev DB).
- `pnpm --filter @workspace/db run push` — push schema changes (dev only).
- `pnpm --filter @workspace/scripts run create-admin` — create an admin user.
- `pnpm --filter @workspace/scripts run mint-token -- --label "PC name" --max-uses 1 --expires-days 7` — mint an enrollment token from the CLI.
- `cd agent && python -m pip install -r requirements.txt && python agent.py` — run the Python agent.

Required environment

- `DATABASE_URL`
- Object storage: `DEFAULT_OBJECT_STORAGE_BUCKET_ID`, `PRIVATE_OBJECT_DIR`, `PUBLIC_OBJECT_SEARCH_PATHS`
- `SESSION_SECRET`
- GitHub connection (for the Downloads feature)

Manage all secrets through the platform's environment/secret tooling — never hardcode them.

Routing

A global reverse proxy routes by path. Always reach services through the proxy (`localhost:80/api/...` in dev), never the service port. Published apps are served over HTTPS on the domains in `$REPLIT_DOMAINS`.

Agent packaging & CI

Installers are built in GitHub Actions (`.github/workflows/build-agent-installers.yml`) on `windows-latest` (PyInstaller + Inno Setup → `.exe`), `macos-latest` (PyInstaller + DMG), and `ubuntu-latest` (PyInstaller → `.tar.gz`), triggered by an `agent-v*` tag, and attached to a GitHub Release. The agent cannot be cross-compiled from Linux. See `agent/README.md`.

9. Operational gotchas

- **API dev workflow has no watch** — restart it after changing server code.
 - **drizzle-kit push** prompts interactively on column renames even with `--force`; on an empty dev DB, drop the affected tables and re-push. It also does **not** diff a partial index's `WHERE` predicate — after editing only a partial `uniqueIndex`'s `.where(...)`, verify via `pg_indexes` and DROP/CREATE by hand, and add a matching `targetWhere` to any `onConflictDoUpdate`.
 - **Stale lib declarations** — after editing `lib/db`, run `pnpm run typecheck:libs` (or `typecheck`) so the API server sees rebuilt types; missing `@workspace/db` exports usually mean stale declarations, not bad imports.
 - **Drizzle wraps driver errors** — the pg error code (e.g. FK 23503) is under `error.cause`, not `error.code`.
 - **No console.log in server code** — use `req.log` in handlers and the singleton `logger` elsewhere.
-

10. Related docs

- `USER_MANUAL.md` — admin/operator guide.
- `API.md` — full HTTP contract for both API surfaces.
- `agent/README.md` — agent setup, platform notes, PyInstaller packaging.
- `lib/api-spec/openapi.yaml` — machine-readable Admin API contract.